

Efeitos temporários

Seis passos: o método que aplica um bônus com prazo, o gancho que o desfaz sozinho, um item que usa os dois, e o mesmo padrão de novo para defesa.

DEPOIS DA EXPLICAÇÃO DE HOJE.

Este documento não repete o código já mostrado. Ele diz o que construir — quem escreve é você.

Onde você está

Você acabou de ver, na explicação, como `aplicar_bonus_ataque_temporario` aplica e lembra um bônus, como `iniciar_turno` desconta e desfaz esse bônus sozinho, e como o `ElixirDeFuria` liga os dois ao jogo. Agora é a sua vez de escrever tudo, sozinho, no seu próprio `personagem.py`, `itens.py` e `main.py`.

Confira que você tem isto antes de seguir:

ARQUIVO	O QUE PRECISA TER
<code>personagem.py</code>	Personagem com <code>nivel</code> , <code>subir_nivel</code> e <code>melhorar_ataque</code> / <code>defesa</code> / <code>vida_maxima</code> já funcionando.
<code>itens.py</code>	Item, PocaoDeVida, Bomba, PocaoDeForca, AnelDeProtecao e Inventario.
<code>main.py</code>	A classe Fase, a lista <code>fases</code> e <code>subir_nivel</code> chamado a cada vitória.

SE O SEU JOGO NÃO ESTIVER NESSE PONTO

Não perca a aula de hoje consertando a de ontem. Pegue os quatro arquivos prontos em `Codigo Base/`, na mesma pasta deste PDF, e comece os exercícios a partir dali.

Nenhum arquivo novo entra hoje. Os métodos moram dentro do `personagem.py` que já existe, o item novo mora dentro do `itens.py` que já existe, e a troca de prêmio é uma linha dentro da lista `fases` que já existe no `main.py`.

EXERCÍCIO 1

aplicar_bonus_ataque_temporario

Crie o método `aplicar_bonus_ataque_temporario(self, quantidade, turnos)` em `Personagem`, no `personagem.py`.

O QUE ELE PRECISA TER

- No `__init__`, acrescente `self._bonus_ataque = 0` e `self._turnos_bonus_ataque = 0`.
- O método recebe `quantidade` e `turnos`, além do `self`.
- Soma `quantidade` em `self.ataque`, imediatamente.
- Guarda `quantidade` em `self._bonus_ataque` e `turnos` em `self._turnos_bonus_ataque`.
- Imprime uma mensagem avisando que o ataque aumentou (o texto exato é com você).

CHECKPOINT 1

O método `aplicar_bonus_ataque_temporario` existe em `Personagem`.

Chamar ele soma o ataque na hora, e guarda os dois números para depois.

TESTE DO CHECKPOINT 1

```
h = Guerreiro("Thoric")
print("antes:", h.ataque)
h.aplicar_bonus_ataque_temporario(6, 3)
print("depois:", h.ataque)
```

Resultado esperado (a mensagem pode ter texto diferente, os números não):

```
antes: 18
Thoric sente o ataque aumentar por um tempo
depois: 24
```

Se o número não mudar: confira se o método está escrevendo `self.ataque`, com `self.` na frente.

Descontando o bônus

Agora ensine `iniciar_turno` a desfazer o bônus quando o prazo acabar.

O QUE ELE PRECISA FAZER, ALÉM DO QUE JÁ FAZIA

- Continua zerando `self.defendendo`, como sempre.
- Se `self._turnos_bonus_ataque` for maior que zero, desconte um turno.
- Se, depois de descontar, a contagem chegar a zero: subtraia `self._bonus_ataque` de `self.ataque`, e zere `self._bonus_ataque`.

CHECKPOINT 2

`iniciar_turno` desconta um turno de bônus por chamada, e só desfaz a soma quando a contagem chega a zero — não antes.

TESTE DO CHECKPOINT 2

```
h = Guerreiro("Thoric")
h.aplicar_bonus_ataque_temporario(6, 3)
for _ in range(3):
    h.iniciar_turno()
print("ataque:", h.ataque)
```

Resultado esperado:

```
Thoric sente o ataque aumentar por um tempo
Thoric sente o ataque voltar ao normal
ataque: 18
```

Se o ataque voltar ao normal na primeira chamada: falta o `if` que confere se a contagem chegou a zero — o código está desfazendo o bônus todo turno, não só no fim.

Se o ataque nunca voltar: confira se `self._turnos_bonus_ataque` está mesmo sendo descontado dentro de `iniciar_turno`.

ElixirDeFuria

Crie a classe `ElixirDeFuria` no `itens.py`, do lado da `PocaoDeForca`.

O QUE ELA PRECISA TER

- Herdar de `Item`.
- `__init__(self)` chamando `super().__init__("Elixir de fúria")`, e guardando `self.forca = 6` e `self.duracao = 3`.
- `usar(self, dono, inimigo)` que chama `dono.aplicar_bonus_ataque_temporario(self.forca, self.duracao)`, e imprime uma mensagem avisando.

CHECKPOINT 3

A classe `ElixirDeFuria` existe no `itens.py`.

`usar` chama `aplicar_bonus_ataque_temporario` — não soma o ataque direto, como a `PocaoDeForca` faz.

TESTE DO CHECKPOINT 3

```
h = Guerreiro("Thoric")
print("antes:", h.ataque)
ElixirDeFuria().usar(h, None)
print("depois:", h.ataque)
```

Resultado esperado (a mensagem pode ter texto diferente, os números não):

```
antes: 18
Thoric sente o ataque aumentar por um tempo
Thoric bebeu o elixir de fúria
depois: 24
```

Se o número bater mas o bônus nunca sumir depois: confira se `usar` chama `aplicar_bonus_ataque_temporario` e não `self.ataque +=` direto no item, ou `dono.ataque +=` — sem passar pelo método certo, ninguém vai lembrar de desfazer.

O elixir no jogo

No `main.py`, troque o `premio` de pelo menos uma fase da lista `fases` para `ElixirDeFuria()`.

CHECKPOINT 4

Pelo menos uma fase dá `ElixirDeFuria` de prêmio.

O jogo roda do início ao fim sem erro na tela.

TESTE FINAL

Jogue até vencer a fase que dá o elixir. Abra a mochila e beba ele. Confira:

- o ataque aumenta na hora;
- o ataque continua aumentado por dois turnos inteiros;
- no terceiro turno seguinte, o ataque volta sozinho ao valor de antes, sem você fazer nada.

EXERCÍCIO 5

O mesmo padrão, para defesa

Agora repita o exercício 1 e o exercício 2, sozinho, pensando em `defesa` em vez de `ataque`. Antes de escrever, responda: quantos nomes precisam mudar?

O QUE PRECISA EXISTIR

- No `__init__`: `self._bonus_defesa = 0` e `self._turnos_bonus_defesa = 0`.
- `aplicar_bonus_defesa_temporario(self, quantidade, turnos)` — mesma forma do método do exercício 1, trocando `ataque` por `defesa`.
- `iniciar_turno` ganha um segundo bloco, do mesmo formato do primeiro, cuidando de `_turnos_bonus_defesa` e revertendo `defesa`.

CHECKPOINT 5

`aplicar_bonus_defesa_temporario` existe e funciona como o de ataque, mexendo em `defesa`.

`iniciar_turno` desconta e desfaz os dois bônus de forma independente — usar um não deveria afetar o outro.

TESTE DO CHECKPOINT 5

```
h = Guerreiro("Thoric")
h.aplicar_bonus_defesa_temporario(4, 2)
print("defesa:", h.defesa)
h.iniciar_turno()
print("defesa:", h.defesa)
h.iniciar_turno()
print("defesa:", h.defesa)
```

Resultado esperado (as mensagens podem ter texto diferente, os números não):

```
Thoric sente a defesa aumentar por um tempo
defesa: 12
defesa: 12
Thoric sente a defesa voltar ao normal
defesa: 8
```

Se a defesa voltar ao normal cedo demais ou tarde demais: confira se você está usando `_turnos_bonus_defesa` nesse segundo bloco, e não reaproveitando `_turnos_bonus_ataque` por engano.

Um item para o bônus de defesa

Crie um item novo que usa `aplicar_bonus_defesa_temporario`. O nome, o valor de `força` e a `duracao` são escolha sua — desde que sigam o mesmo contrato dos outros itens.

O QUE ELE PRECISA TER

- Herdar de `Item`.
- `__init__` guardando um nome (à sua escolha) e os dois números do efeito.
- `usar(self, dono, inimigo)` chamando `dono.aplicar_bonus_defesa_temporario(...)`.

Depois, coloque ele como prêmio de outra fase — diferente da que já dá o `ElixirDeFuria`.

CHECKPOINT 6, O ÚLTIMO

O item novo existe, usa `aplicar_bonus_defesa_temporario`, e é prêmio de pelo menos uma fase.

O jogo roda do início ao fim com os dois efeitos temporários funcionando — um de ataque, um de defesa — sem interferir um no outro.

Antes de encerrar, confira

- ☐ `aplicar_bonus_ataque_temporario` e `aplicar_bonus_defesa_temporario` existem em `personagem.py`
- ☐ `iniciar_turno` desconta e desfaz os dois bônus de forma independente
- ☐ `ElixirDeFuria` existe em `itens.py` e chama `aplicar_bonus_ataque_temporario`
- ☐ Seu item de defesa existe e chama `aplicar_bonus_defesa_temporario`
- ☐ Os dois itens são prêmio de fases diferentes
- ☐ O jogo roda do início ao fim sem erro na tela